NewsDC application study and client presentation guide

Prepared 7 November 2026. This guide explains the implementation, the engineering problems it addresses, and how to discuss it with a client. Read the product story first, then the feature explanations, then practise the questions without looking at the answers.

1 The product story

NewsDC collects data center articles, research, discussions and public video metadata into one searchable library. It helps a reader discover relevant material, understand supported summaries, save a reading list and ask questions with links to indexed evidence. Operators manage collection and AI processing from a protected dashboard.

Client introduction: "We built a data center intelligence application that brings permitted public content into one library. It normalizes and deduplicates incoming records, enriches eligible text with AI, and lets readers search, personalize their feed and ask evidence-based questions. Original sources remain visible. The application runs locally in Docker; public deployment and final dataset acceptance remain release work."

The key distinction is between collecting a record and verifying its relevance. The latest recorded handoff had 702 source records: 600 research papers, 50 articles, 50 discussions and two videos. That is a historical snapshot, not a live count or proof of 500 AI-confirmed relevant records. Check Dashboard immediately before presenting numbers.

2 Architecture and technology

The application is a modular monolith: one Express backend contains separate ingestion, content, AI and operations modules. React runs in the browser. PostgreSQL holds shared data. The production Express process serves both the compiled frontend and the API. Background work currently runs inside the same backend runtime, not a distributed worker cluster.

The main flow is: approved API or RSS source -> adapter -> validation and normalization -> deduplication -> PostgreSQL -> optional AI enrichment -> persisted output -> API -> React screen. The assistant uses a separate flow: question -> PostgreSQL retrieval -> bounded evidence -> model -> citation validation -> cached answer.

- React and JavaScript/JSX implement components, state and screen interactions.
- Vite provides local frontend development and the production build. It is not the production database or API server.
- Tailwind CSS is integrated, with global CSS and theme variables supplying much of the visual design.
- Lucide React provides icons. React Router maps URLs to pages and supports navigation and query parameters.
- Node.js and Express 5 provide the REST API, validation boundaries and orchestration.
- Prisma Client 7 with the PostgreSQL driver adapter performs application queries and transactions. PostgreSQL 16 provides persistence, constraints, relationships and full-text search.

- Zod validates user input and model output at runtime. JavaScript alone does not guarantee those contracts.
- fast-xml-parser maps RSS and Atom responses into source candidates.
- Gemini and OpenAI are accessed through server-side HTTP calls behind a shared provider gateway. There is no LangGraph or vector database dependency.
- Docker and Compose package the application and PostgreSQL. npm workspaces connect the frontend, API, database and shared taxonomy packages. Node's test runner and Prettier support verification and formatting.

Why this architecture: it keeps modules understandable without the operational overhead of microservices. A separate database gives persistence and query capabilities; browser-local reader state avoids requiring registration. The tradeoff is that process-local locks and ranking are intended for a modest MVP, not multiple independent workers at large scale.

3 Where data lives

PostgreSQL stores the shared content library. Each content item has a UUID, format, title, description, original and canonical URL, optional author/date/image/duration/body, publisher and source references, processing state and timestamps. ARTICLE, VIDEO, RESEARCH and DISCUSSION use the same model. Missing metadata stays null rather than being fabricated.

Publishers describe the originating domain; sources describe the collection adapter and endpoint. A Hacker News discovery can point to an external publisher, so source and publisher must remain separate concepts. Categories are the controlled 14-topic taxonomy. Tags are source-provided labels. Mapping tables allow many topics and tags per content item without duplicating item records.

AI fields store summary, insights, intelligence type, relevance flag and confidence, relevance and quality scores, model, prompt version and processing time. JSON metadata holds flexible provenance such as input hashes, evidence excerpts, reasons, provider and attempts. Structured columns remain available for filtering and aggregation.

Operational tables store ingestion runs, processing logs, provider calls, daily request totals, conservative fallback usage reservations, cached answers/reports and persisted application settings. Schema migration names and checksums track database evolution. A content-relations table exists, but the current related-content endpoint computes shared-topic matches directly; table existence alone does not establish a feature's runtime behavior.

The browser stores bookmarks, preferences, reading history and the theme choice in localStorage. There are no end-user accounts or cross-device synchronization. The operator credential stays in dashboard memory, rather than browser persistence. Assistant screen exchanges are temporary React state; the server separately caches generated answers.

Privacy explanation: "Your reader preferences are stored locally, but the app sends selected topic/type preferences and read item IDs with recommendation requests to calculate the feed. Selected source excerpts and questions are sent to the configured model provider for AI work. Local storage does not mean every interaction stays exclusively on the device."

4 Ingestion and source discovery

Approved sources and formats

The current registry includes Hacker News through Algolia's public API, Data Center Knowledge RSS and arXiv research metadata. YouTube links discovered through the HN adapter can be upgraded with public YouTube oEmbed metadata. Operators can also import bounded metadata batches against an existing source. Readers cannot make the backend fetch arbitrary URLs.

HN provides discussion discovery and available points/comments. RSS provides article metadata. arXiv Atom supplies title, abstract, authors, category tags and available DOI/journal metadata. Research ingestion does not download PDFs. YouTube discovery provides available title/channel/thumbnail; it does not download a video or transcript. A discovery timestamp is not substituted for an unknown video publication date.

How one run works

An operator requests ingestion or the opt-in scheduler triggers it. The service creates persistent run IDs, then processes enabled sources sequentially. Each adapter returns candidates in a common shape. Validation checks required title, HTTP(S) URL, supported format, dates and optional fields. Normalization removes markup, collapses whitespace and bounds text. Initial topic rules are keyword rules, explicitly distinct from later AI classification.

The service canonicalizes URLs by removing fragments and selected tracking parameters, sorting query parameters and preserving meaningful URL details. A SHA-256 fingerprint of normalized title plus domain catches some same-title/domain duplicates. Database unique constraints provide a second defense against competing inserts. This is conservative duplicate prevention, not universal semantic duplicate detection across publishers.

Existing matches are refreshed rather than reinserted. Source input changes invalidate dependent AI output; unchanged input can retain it. Tags update atomically: omitted tags preserve existing mappings, while an explicit empty list clears them. Tags are normalized, deduplicated and bounded, and remain separate from AI topics.

Every run records fetched, inserted, skipped and failed counts plus status and safe error details. Individual candidate failures do not require discarding every successful item. A run may contain inserted items and still be marked failed when some candidates fail. Scheduled and manual runs share a process-level guard against overlap.

Problems and solutions

Different sources have different field shapes: adapters isolate those differences. Repeated refreshes could create duplicates: canonical URLs, fingerprints and unique constraints prevent many duplicates. Malformed records could poison storage: Zod and database checks reject invalid values. Missing data could mislead readers: optional fields remain absent and the UI labels unavailable dates and text scope.

Unbounded fetches could exhaust resources or access unintended hosts: the source registry, no redirects, timeouts and response limits bound requests. Generic source fetches use a 10-second timeout and a two-million-byte response bound, with at most two attempts; selected permanent/rate-limit responses do not trigger futile immediate retries. arXiv requests are serialized with at least three seconds between them within the process.

Research backfill accepts reviewed pages of 50 records, offsets 0 to 550. It preserves progress and respects deduplication and pacing. Historical verification recorded 550 new papers, 50 duplicates skipped and zero failures across 12 pages. This is evidence of that run, not a guarantee for every future upstream response.

Scheduler settings are independent of AI settings. Scheduled ingestion defaults off; when enabled, its default interval is 30 minutes, with a one-minute minimum. Run one ingestion/API process in this MVP because its coordination and source pacing are not distributed locks.

5 AI generation and enrichment

Generation explained simply

Generation means sending bounded source evidence plus explicit instructions and an output schema to a language model, then checking the returned structure before using it. NewsDC does not train its own model. Ingestion supplies the evidence; the provider produces proposed analysis; the application validates and persists it.

There are three distinct generation tasks: content enrichment, assistant answers and trend summaries. Ordinary feed, search and detail reads reuse stored data and do not generate summaries again.

Eligibility and evidence

Eligible source text is the supplied body when available, otherwise the description or research abstract, bounded to 6,000 characters. The title is bounded to 1,000. Other than the special video rule, at least 160 characters and 25 words are required. Thin records are skipped without a provider call. Video enrichment requires stored text and an explicit permitted-transcript flag; the current public video discovery does not provide transcripts.

Research with only an abstract is labelled abstract-only. Article source descriptions are not represented as full article bodies. The system splits normalized evidence into bounded excerpts and assigns IDs such as E1. The model selects allowed IDs to support its summary and insights; the application resolves them to original excerpts and checks the resulting quotes occur in the supplied text.

The schema permits a relevance flag and confidence, up to five unique configured topics, one controlled intelligence type, a concise summary, up to three insights, estimated scores and explanations. Intelligence type distinguishes news, technical research, investment intelligence, engineering ideas and other material; it is separate from the article/video/research/discussion format.

Relevance score estimates directness and depth of data center relevance from 0 to 100. Quality score estimates the detail, evidence and usefulness of the supplied text from 0 to 100. Confidence is a separate 0-to-1 field. These are model-assisted estimates, not objective truth, publisher reputation ratings or measured scientific accuracy.

Irrelevant or insufficient evidence must have no generated summary, insights or quality score. An irrelevant record remains available to operations in the database, but public feed/search/related listings and visible topic counts exclude records marked irrelevant. Unclassified records are still eligible for public discovery; visible does not mean AI-confirmed relevant.

Provider routing and safeguards

The source currently configures Gemini model gemini-3.5-flash-lite and fallback model gpt-5.4-mini. These are code values observed in this review, not a promise of provider availability or a permanent business decision. Recheck them before presenting a model-specific commitment.

The gateway makes an initial Gemini attempt and up to two retries for temporary failures, then at most one OpenAI fallback attempt. Retry delay is normally two then four seconds, adjusted for supported Retry-After values. Permanent failures avoid futile retries. Safety refusals and oversized input do not fall through to another provider. Daily provider quota detection avoids a repeated quota loop.

Each provider HTTP attempt has a 20-second timeout. Output is limited to 1,024 tokens, serialized request size is bounded, and response reading is capped. These are per-attempt bounds; an end-to-end task can take longer because of retries, backoff and interactive waiting. One shared process-level guard permits one active provider generation at a time across tasks.

Keys remain server-side. The gateway records an attempt before sending and finishes it with status, error and available token usage. If durable accounting fails before the call, it fails closed. Application spending caps and fixed daily AI request caps were explicitly removed. Provider quotas, public traffic throttling, payload bounds, concurrency guards and bounded retries remain. A fallback cost figure is conservative accounting based on code assumptions, not the provider invoice or a newly verified price quotation.

Persistence and stale output

Processing moves through pending, processing, completed, failed or skipped states. An operator can queue one to ten records. The runner processes sequentially and stores logs. Output, topic mappings and completion status are committed transactionally. Processing ownership checks prevent another task's output from being overwritten.

The cache/currentness identity combines prompt version and a hash of the actual model input. Unchanged successful input reuses saved enrichment. If source text, date or scope changes, dependent fields are cleared. If source input changes while generation is running, that stale result is discarded. Updating unrelated engagement metadata does not force another generation.

On restart, interrupted work is marked failed rather than falsely completed. Prompt-version changes invalidate old output. Public serialization hides non-current generated values. Failed items need explicit retry; the automatic loop is not an unbounded retry engine.

Automatic processing

AI_AUTO_ENRICHMENT is an independent opt-in flag. The loop normally checks every five seconds, honors persisted pause/resume state, avoids active interactive/provider work and takes pending candidates in a bounded window. It processes eligible work incrementally and marks thin inputs skipped. Post-ingestion hooks wake the loop. It can refresh trend reports after pending work drains. This is in-process automation with persistent records, not BullMQ or a durable distributed queue.

Reliability boundary

Structured schemas, evidence IDs, citation checks and explicit insufficiency reduce unsupported output. They do not prove every sentence follows logically from its evidence. Quote presence is a structural grounding check, not a semantic fact checker. Human review against original sources remains part of acceptance.

6 Reader features and how to demonstrate them

Discover and personalization

The home screen calls the For You endpoint unless an explicit sort selects the general content listing. Saved topic choices, preferred formats and read item IDs are included in recommendation arguments. Preferences boost matching items rather than hiding all other subjects. Explicit filters narrow the candidate set first.

For You score = 200 for any preferred-topic match + 100 for a preferred-format match + 0.10 times freshness + 0.02 times engagement - 20 if read + current AI bonus. The topic bonus is a boolean match, not 200 per matching topic. AI bonus = 0.40 times relevance score + 0.25 times quality score, only for current enrichment.

Freshness = 100 times exp(-age in days / 7), with negative age clamped to zero. Available points become min(100, 15 times log(1 + nonnegative points)); absent points contribute zero. This is a deterministic heuristic, not a learned recommendation model. Candidates are the newest 1,000 matching records, ranked in Node and then paginated with deterministic tie-breakers.

Demonstrate by saving a topic and format preference, returning to Discover and explaining changed ordering. Avoid claiming the algorithm searches every historical record once the matching collection exceeds 1,000.

Trending and emerging themes

Trending score = 0.80 times freshness + 0.20 times available engagement + current AI bonus. It supports timeframe filters and continuous rank numbering across pages. Missing engagement does not become invented views, likes or popularity.

The emerging-theme report is different from item ranking. It compares category counts in the latest seven days with the preceding seven, considering up to 1,000 date-eligible records. Topics need at least two current items and positive growth; up to three themes are selected. A model summarizes supplied examples with allowed source IDs. These are trends in the indexed collection, subject to coverage bias, not proof of industry-wide growth.

Reports are cached for one hour with day/source-sensitive keys. Public reads retrieve status and existing reports; generation uses protected refresh or automation. Reopening Trending does not itself demand a new model generation.

Topics and related content

Fourteen controlled topics are shared across frontend/backend and stored in PostgreSQL. Topic counts come from mapped, publicly eligible items. Clicking a topic adds a filter. AI topics replace initial keyword labels after successful enrichment; source tags remain separate. Related content returns up to five other eligible items sharing topics, ordered by recency. It is category matching, not vector similarity.

Search and filters

Search uses PostgreSQL full-text matching through Prisma over title, description, supplied text and stored summary. Query words are normalized and bounded; ordinary search combines terms with AND across a matched field. The assistant has its own more permissive retrieval query and reranking. This is keyword retrieval, not embeddings or semantic vector search.

Filters combine content type, topic, publisher, dates and minimum quality/relevance. Sort choices include date, title, quality and relevance. Query parameters preserve state in the URL, making refresh and back navigation predictable. Null scores do not satisfy minimum-score filters, so a scored-only query can have few results while classification is pending.

Search relevance sorting with a query uses text relevance, whereas score filtering uses stored AI relevance. Do not treat those two meanings as interchangeable. A generated search-vector GIN index exists in schema, but the app uses Prisma full-text APIs; index use and performance must be measured rather than assumed.

Pagination and exports

The UI requests ten records per page. The API validates positive pages and limits of at most 100. Ordinary queries use database skip/take and count; recommendations paginate their bounded ranked candidate set. Stable ID tie-breakers and last-valid-page recovery address duplicate ordering and impossible page numbers.

JSON and Markdown exports preserve applicable filters and return at most 100 matches from the start of the filtered set. JSON includes structured content/AI information; Markdown currently contains title, format, publisher, description and original link. Markdown export is not an export of every AI insight. Export order follows the general listing, not necessarily personalized feed ranking, and is not just the currently visible page.

Content detail

Detail loads an item by UUID and separately requests related records. It shows available metadata, publisher or discovery description, source tags, original source action, supported research/video fields and current AI output. Original content opens at the publisher; NewsDC is not hosting full third-party articles or video playback.

Scores and supporting evidence live in expandable detail rather than default feed badges. AI summary labels, model/provenance and pending/failed/skipped states distinguish generated information from publisher text. Successfully loaded details record local reading activity; unsuccessful requests do not count as successful reads.

Saved items and reading history

Bookmarks store content IDs locally. Updates commit in React only when the storage write succeeds; failure gives feedback. New additions are limited to 200. Legacy larger lists remain accessible in groups. Reloading reuses IDs to retrieve current database content, not a full offline copy. Clearing site data removes these local choices.

History keeps the most recent 200 unique item openings with timestamps. Reopening moves the item to the front without duplicates. Users can revisit or clear it; unavailable legacy entries have a recovery presentation. History also supplies read penalties for personalization. It does not track time spent reading, completion or detailed behavioral analytics.

Preferences and theme

Preferences allow 14 topic choices and four formats. Editing uses a draft: Save commits, Cancel discards changes and Reset clears the draft until saved. Unknown or duplicate stored values are normalized. Failed writes do not silently replace persisted preferences.

Light/dark mode changes shared CSS variables and is saved locally. Desktop uses a sidebar and search header; narrow layouts adapt navigation and content spacing. Semantic controls, focus indicators, labels, reduced-motion styling and loading/error/empty states support accessibility. Recorded checks include 320px layouts and selected contrast pairs, but are not full WCAG certification or a complete device matrix.

The resource hook aborts stale requests on dependency changes or unmount and provides retry state. This addresses fast-filter/navigation races where an older response could otherwise replace newer screen data. Ordinary keyboard controls work; custom keyboard shortcuts remain P2.

7 Assistant questions and citations

The assistant accepts a 3-to-500-character question with optional type/topic/date filters. It does not browse the live internet. It extracts up to eight meaningful query groups, removes common question words and expands selected aliases such as cooling/thermal/liquid and GPU/accelerator.

Prisma full-text retrieval excludes explicitly irrelevant content, applies filters and retrieves up to 80 candidates. Application reranking rewards query-group matches in titles more than body text. It rejects insufficiently long excerpts or unsafe URLs and takes up to five sources, each with a bounded 1,000-character excerpt and evidence-scope label.

If retrieval finds no usable evidence, the application returns insufficient evidence without generating an answer. Otherwise it asks the model for up to four paragraphs with source IDs from that exact source set. Every factual paragraph must have citations. Unknown citation IDs, model-supplied URLs and inconsistent answer/insufficiency shapes fail validation. The server resolves IDs to stored content IDs and trusted original URLs, and returns source cards.

Answers are cached for 24 hours using question, filters, prompt version and retrieved evidence. Repeated unchanged questions can reuse the answer across server restarts. Changed retrieval evidence produces a different key. The UI keeps up to five exchanges in memory, but each backend request is a standalone question: previous exchanges are not sent as conversational context. Follow-up pronouns therefore should be replaced with self-contained questions.

An interactive question can briefly wait for a provider slot, but concurrency remains bounded and busy/failure states are explicit. The screen keeps a failed question for retry. It can return insufficient evidence even with matching source cards if the evidence does not answer the actual question.

Client phrasing: "Our assistant retrieves evidence from our indexed collection and asks the model to synthesize a cited answer. We validate references against the retrieved set and construct links from stored records. We deliberately show insufficient evidence when coverage is weak. This reduces fabricated references, but final claim quality still needs source review."

8 Dashboard and operational security

The dashboard is a protected operator tool, not an end-user login system. Backend bearer-key authorization protects metrics, source toggles, imports, ingestion, backfill, provider probes, enrichment, automatic processing and trend refresh. Hiding a button alone is not the protection. Valid operator credentials remain usable even after invalid-attempt throttling.

Metrics include total/type counts, processing counts, confirmed relevant/excluded/unclassified coverage, recent runs and recent created-record activity. The daily chart is based on records' creation dates; it is not a count of every upstream fetch or an AI accuracy chart. Provider controls show enabled/configured state,

attempts and usage without exposing keys.

Defenses include runtime request validation, bounded JSON bodies, safe error envelopes, request IDs, structured logs, approved source endpoints, no redirects, provider timeouts, Prisma queries and React text escaping. Browser headers include CSP, framing denial, nosniff and no-referrer. Original links use safe new-tab attributes.

Public non-admin/non-health API traffic is limited per process and socket IP to 120 requests per minute. That traffic control also applies to assistant requests; it is different from the removed daily AI caps. Admin invalid-attempt state and public limiter maps are bounded. These measures are not a complete security assessment. Proxy identity, TLS, distributed limits, backups and production monitoring require hosting-specific work.

9 Packaging and failure recovery

Compose runs two services: the combined app and PostgreSQL. The Docker image installs workspace dependencies, generates Prisma Client and builds Vite. It runs as a non-root node user. Startup applies versioned migrations before starting Express. The app waits for database health; app readiness checks database/migration state.

The local app binds host loopback port 5000; the database is exposed on loopback port 55434 by default. PostgreSQL data lives in a named volume and survives ordinary container recreation. That is persistence, not a backup. Deleting the volume would delete the stored collection.

Migrations use transactions, an advisory lock and checksums. Applied migrations must not be rewritten; add a new migration and regenerate the Prisma model/client. Raw SQL is confined to schema migration machinery and database-specific test infrastructure. Ordinary application queries use Prisma.

Liveness indicates the process is running. Readiness indicates database and migrations are usable. Database outages return safe 503 responses where recognized; unexpected failures return safe 500 responses without exposing credentials. Interrupted runs are recovered as failed, allowing honest operator retry. Graceful shutdown stops timers and closes database connections.

10 Engineering problems to explain

These are demonstrated design problems and implemented solutions. Describe them as personal debugging incidents only when supported by the implementation history or your own recollection.

- Duplicate ingestion: repeated URLs/tracking links inflated counts. Canonicalization, title/domain fingerprints and database uniqueness make repeat insertion idempotent within their defined match rules.
- Unreliable model structure: free text could not reliably drive UI or database fields. JSON schemas, Zod validation, enum bounds and persistence-boundary revalidation reject malformed results.
- Fabricated evidence links: models could invent URLs. Models select source IDs and the server resolves stored URLs; invalid IDs and generated URLs are rejected.
- Weak content: titles or thin descriptions cannot support rich summaries. Eligibility checks and scope labels skip unsupported work and distinguish abstracts from full text.
- Stale summaries: refreshed source data could leave outdated AI claims. Input hashes, versioning and ownership checks invalidate or discard stale output.

- Duplicate paid work: repeated reads/questions could regenerate the same answer. Persisted enrichment and evidence-sensitive caches reuse current output.
- Provider outages and quotas: one provider failure could disable generation. Bounded retry/backoff and one fallback improve availability without infinite loops; refusals do not bypass safety through fallback.
- Competing background and interactive work: shared provider guards and busy states bound concurrency. Automatic processing avoids active interactive work; multiple independent backend processes remain outside the guarantee.
- Interrupted jobs: a crash could leave permanent running badges. Startup recovery marks interrupted work failed and retains logs.
- Local storage failure: blocked/corrupt browser storage could crash UI or falsely show a saved item. Normalizers, catches and commit-after-success handling recover or show feedback.
- Unbounded ranking: sorting every stored row could grow expensive. The app ranks at most 1,000 recent matching candidates and exposes that limitation.
- Pagination edge cases: ties, old page numbers and removal of the last item could show confusing results. Stable tie-breakers, rank offsets and last-valid-page recovery improve behavior.
- Export mismatch: exports could omit the selected timeframe. Filter arguments are preserved; export limits and ordering still need clear explanation.
- Operator lockout: invalid-request throttling previously also blocked valid keys. Authorization now preserves valid operator access while throttling invalid attempts.
- Stale frontend responses: quick navigation could show old data. Abortable resource fetching avoids applying abandoned requests.
- Environment problems: historical logs record Docker/WSL engine recovery and sandbox loopback restrictions. These are development environment issues, not news ingestion or AI algorithm features.

11 Client question bank

Product and data questions

Question 1: What problem does it solve?

Answer: It reduces fragmented data center research by collecting, searching and summarizing permitted source material with attribution.

Question 2: Where does the news come from?

Answer: The approved HN API, Data Center Knowledge RSS and arXiv metadata, with HN-discovered YouTube oEmbed metadata.

Question 3: Do you scrape entire websites?

Answer: Current adapters fetch bounded public API/feed metadata; they do not download arbitrary article bodies, PDFs or video transcripts.

Question 4: How do you store it?

Answer: Shared relational records and mappings in PostgreSQL, queried by Prisma; local reader state stays in browser storage.

Question 5: Why PostgreSQL?

Answer: It combines transactions, relationships, constraints and full-text search in one operational database.

Question 6: Why Prisma?

Answer: It provides structured application queries and transactions while versioned migrations retain database-specific constraints and indexes.

Question 7: How are duplicates prevented?

Answer: Canonical URL and title/domain fingerprint checks plus unique indexes; it is not universal cross-publisher semantic deduplication.

Question 8: Are all records relevant?

Answer: No. Metadata import, unclassified content and AI-confirmed relevance are different states; Dashboard separates them.

Question 9: Have you reached 500 relevant records?

Answer: Use current confirmed-relevant metrics. The recorded 702 total alone does not meet that requirement.

Question 10: What if dates or images are missing?

Answer: They remain absent; the interface uses explicit missing-data labels or image fallback behavior.

Question 11: Is content updated automatically?

Answer: Only when opt-in ingestion is enabled; source enablement and run history are visible to operators.

Question 12: Can we add another source?

Answer: Add and review an adapter/registry entry with metadata mapping, request bounds and tests; arbitrary reader URLs are not supported.

AI and evidence questions

Question 13: How does generation work?

Answer: A server sends bounded source evidence and an output schema to a provider, validates the response and stores supported output.

Question 14: Did you train the AI?

Answer: No. The application orchestrates hosted models with prompts, retrieval, validation and persistence.

Question 15: Which provider is primary?

Answer: Gemini first; up to two temporary-failure retries, then one OpenAI fallback. Check current code/config before naming model commitments.

Question 16: What if both providers fail?

Answer: The task fails explicitly, logs remain and existing metadata stays usable; no simulated answer is substituted.

Question 17: How do citations work?

Answer: Models select allowed retrieved IDs; the server maps those IDs to real stored URLs and rejects invented references.

Question 18: Does citation validation guarantee truth?

Answer: No. It ensures reference membership and structure; semantic support and source reliability still need review.

Question 19: What is RAG here?

Answer: Retrieve indexed PostgreSQL evidence, augment the prompt, generate a constrained cited answer. It does not require a vector database.

Question 20: Does the assistant search the internet live?

Answer: No. It answers from the indexed collection and supplied excerpts.

Question 21: Does it remember earlier questions?

Answer: The UI shows recent exchanges, but backend generation currently receives only the current question and filters.

Question 22: Can it read full research papers?

Answer: Current research evidence is abstract-only unless permitted text was separately supplied; PDFs are not downloaded.

Question 23: Does it understand videos?

Answer: Current discovery is metadata-only; content enrichment requires an explicitly permitted transcript, absent from this discovery path.

Question 24: What does quality score mean?

Answer: An estimated usefulness/detail/evidence rating of supplied text, with a reason; not a certified publisher trust score.

Question 25: When does it refuse to answer?

Answer: When retrieval has no usable evidence or the model reports insufficient support; failures remain explicit.

Question 26: How do you avoid repeated AI costs?

Answer: Stored enrichment, input hashes, 24-hour answer cache and one-hour trend-report cache reuse unchanged output.

Question 27: Is there a spending cap?

Answer: Application spending and fixed daily AI caps were removed by user instruction; provider quotas, bounds and usage visibility remain.

Question 28: What is sent to providers?

Answer: Relevant source fields/excerpts and task instructions; assistant questions also go to providers. Keys stay on the server.

Question 29: How is prompt injection handled?

Answer: Sources/questions are treated as untrusted data, outputs are constrained, and models are not given arbitrary browsing tools; these reduce risk without guaranteeing immunity.

Question 30: Can generation become outdated?

Answer: Yes; input/version currentness checks invalidate changed enrichment and evidence-sensitive keys separate changed answers.

UI architecture and operations questions

Question 31: How is For You personalized without login?

Answer: Browser preferences/history are sent as recommendation inputs and deterministically boost/rerank eligible records.

Question 32: Is it machine-learning recommendation?

Answer: No learned recommender is trained; documented weights combine preference, freshness, engagement and current stored AI signals.

Question 33: How is Trending different?

Answer: It prioritizes recency and available engagement plus AI signals; source-linked theme reports compare indexed category windows separately.

Question 34: Are related items semantic matches?

Answer: Current related items share controlled topics; vector similarity is deferred.

Question 35: Does saving work across devices?

Answer: No. Bookmarks/preferences/history are local to this browser and can be lost when site storage is cleared.

Question 36: Is Saved available offline?

Answer: It stores IDs, then fetches current records; there is no offline library implementation.

Question 37: Why no registration?

Answer: The agreed MVP uses local state to reduce user friction and delivery scope.

Question 38: How is the dashboard protected?

Answer: Every admin request is checked server-side against the operator bearer key; the key is not bundled into the frontend.

Question 39: What happens on a database outage?

Answer: Readiness fails and requests return controlled errors; metadata is not falsely reported as processed.

Question 40: What happens after restart?

Answer: PostgreSQL data/caches/settings persist; interrupted jobs are marked failed, not silently declared successful.

Question 41: Can this scale to millions of items?

Answer: That has not been demonstrated. Ranking is capped at 1,000 candidates; search profiling, distributed queues/limits and measured scaling are future work.

Question 42: Why not Redis or microservices?

Answer: Current bounded single-process needs fit a modular monolith. Add infrastructure when measured workload justifies it.

Question 43: Is it deployed publicly?

Answer: Local Docker delivery is recorded; public hosting, TLS/proxy setup and release acceptance remain outstanding.

Question 44: Is it secure and accessible?

Answer: It has concrete controls and selected checks, but not a complete security or accessibility certification.

Question 45: What are the export limits?

Answer: At most 100 filtered records; Markdown exports descriptions and original links, and ordering is not necessarily the personalized ranking.

Question 46: How do you verify features?

Answer: Unit logic, HTTP/provider tests, isolated PostgreSQL integration, build checks and selected browser smoke checks; report exact executed coverage and skips.

Question 47: What still needs approval or completion?

Answer: Current dataset relevance/AI quality review, user acceptance, hosting choice and production release controls. P2 remains deferred.

Question 48: What would you improve next?

Answer: Choose improvements from observed failures: retrieval recall, source coverage and reviewed summary quality first; then measured performance and deployment readiness.

12 Presentation and study plan

A ten minute demonstration

Minute 1: state the user problem and explain the architecture in one sentence. Minutes 2 and 3: Discover, filters, Topics and original attribution. Minute 4: change preferences and show ranking rather than claiming hidden intelligence. Minute 5: save an item, reload and show local history. Minute 6: open an actually enriched record and inspect evidence, scope and scores. Minute 7: ask a known supported question and open its original citation. Minute 8: show an insufficient-evidence case and explain the coverage boundary. Minute 9: unlock Dashboard privately and show current ingestion/processing/coverage metrics. Minute 10: explain Docker packaging and remaining release work.

Before the meeting, verify the app is ready, check current counts, select a genuinely completed enriched item and preflight source links. Avoid displaying environment files or credentials. Remember that a cached answer and a fresh provider response are different demonstrations. Live generation can be affected by provider

availability and background work.

How to become fluent

Study session one: explain the product, stack and storage without code. Session two: trace one record through normalization, deduplication and persistence. Session three: trace enrichment with a skipped, successful and failed input. Session four: explain retrieval and citations, including an insufficient-evidence answer. Session five: practise the client questions and the ten-minute demo.

For each feature, answer these five prompts aloud: what the user does, what frontend/API/database modules do, which problem the design solves, how it was checked, and what its limit is. If you cannot explain a term plainly, return to that section and trace one real example.

Practice exercise: explain why 702 stored records, 702 visible items and 702 relevant records are three different claims. Then explain why an answer with valid citation IDs can still need factual review. Finally explain why localStorage personalization can still involve sending preference inputs to a server.

13 Verification and evidence ledger

This review inspected source and historical records without triggering ingestion, provider generation, source toggles or database changes. The frontend production build and all 23 browser-state logic tests passed in this review. These logic tests include a historical preview-ranking utility; current production ranking is the backend formula described above, so their existence alone does not prove that formula end to end.

The initial API rerun encountered sandbox loopback EACCES errors on temporary HTTP servers. The rerun with loopback access completed with 38 passed, zero failed and four database-dependent tests skipped because TEST_DATABASE_URL was not configured. Database integration tests require a dedicated TEST_DATABASE_URL and must not use the development collection. Historical handoff evidence records 71 API results with isolated PostgreSQL, seven DB results and 23 frontend results; those historical totals are not a claim that every integration check ran today.

Historical browser evidence covers selected live search/detail/bookmark/preferences/history/theme/operator flows and 320px assistant/video checks. Historical provider evidence records real Gemini cited answers for selected questions and explicit insufficiency for others. Full client acceptance should still compare actual summaries and claims with original evidence.

Older README, deployment and tracker paragraphs contain superseded statements that AI is disabled/pending, tags remain unfinished or the collection has 141 records. The latest assistant/automation handoff and current source supersede those historical descriptions. Do not repeat them as current behavior or treat any historical count as live.

14 Code map for deeper study

- apps/web/src/app/routes.jsx maps the screens; components/layout provides the shared shell.
- apps/web/src/components/content/FeedPage.jsx builds filters, personalization arguments, pagination and export links.
- apps/web/src/context/NewsContext.jsx and hooks/utils handle reader state, storage and theme.
- apps/web/src/pages/AskAI.jsx renders standalone questions, source cards and temporary exchange history.

- services/api/src/app.js and routes.js define HTTP boundaries, errors, headers, authorization and endpoints.
- services/api/src/content/service.js implements query validation, serialization, public eligibility, full-text filtering and ranking.
- services/api/src/ingestion/sources.js, video.js, normalize.js and service.js implement collection, mapping, normalization, deduplication and persistence.
- services/api/src/ingestion/scheduler.js and backfill.js coordinate timed ingestion and reviewed research pagination.
- services/api/src/ai/providers.js and limits.js implement provider transport, retries, validation and usage records.
- services/api/src/ai/enrichment-input.js, enrichment-contract.js and enrichment.js implement eligibility, evidence, hashes and transactional output.
- services/api/src/ai/assistant.js and trends.js implement retrieval/citations and source-linked theme reports.
- services/api/src/ai/automation.js and runtime.js share guards and background lifecycle.
- packages/db/prisma/schema.prisma and database/migrations define persisted contracts; packages/shared/src/taxonomy.js supplies controlled topics.
- services/api/test, packages/db/test and apps/web/src/utils contain automated verification. IMPLEMENTATION_STATUS.md and ASSISTANT_AUTOMATION.md record historical runtime evidence and remaining acceptance.

Requirements context: MVP_SCOPE.md and FEATURES.md define core discovery, ingestion, AI, local-state and delivery requirements; dashboard, themes, exports and research/video enhancements are included P1 work. Custom shortcuts, vector search, user accounts and cross-device synchronization remain outside the delivered MVP scope. This guide adds no application feature or dependency.